CONTENTS

**Cover Page** i

**Table of Contents** i

**I Introduction** 1

**II Theoretical Background** 1
- II-A Why a NoC is Used . . . 1
- II-B Packets, Flits, and Wormhole Behaviour . . . 1
- II-C Mesh and Torus Topologies . . . 1
- II-D Traffic Patterns and Metrics . . . 1

**III Work Progress** 2
- III-A Baseline Bring-Up . . . 2
- III-B Delay Measurement and Wormhole Arbitration . . . 2
- III-C Scaling to a 4x4 Mesh . . . 2
- III-D Traffic Mapping and Mesh Measurement . . . 3
- III-E Torus Conversion and Final Validation . . . 3

**IV Results and Discussion** 4

**V Works Pending** 5

**VI Tentative Plan for Pending Work** 5

**VII Conclusion** 5

**Appendix A: Modified Source Files** 5
- A-A `arbiter.cpp` and `arbiter.h` . . . 5
- A-B `buf_fifo.cpp` and `buf_fifo.h` . . . 7
- A-C `crossbar.cpp` and `crossbar.h` . . . 8
- A-D `router.cpp` and `router.h` . . . 9
- A-E `source.cpp` and `source.h` . . . 10
- A-F `sink.cpp` and `sink.h` . . . 10
- A-G `packet.h` . . . 11
- A-H `main_noc.cpp` . . . 11

**References** 14

# Final Report on SystemC-Based Network-on-Chip

Ramtin Soleymani
Toronto, Canada

***Abstract*—This final report presents the completed SystemC Network-on-Chip (NoC). The work began from the provided 1x2 learning baseline and was extended into a 4x4 mesh and a 4x4 torus, each with sixteen routers and sixteen local source/sink pairs. The completed simulator supports timestamp-based delay measurement, wormhole-style route reservation in the arbiter, one-to-one uniform and neighbouring traffic mappings, waveform tracing, and end-of-run checks for packet count, flit count, destination correctness, and timeout behaviour. Final measured results show that all required runs complete successfully. The mesh uniform run produced an average packet delay of 25.62 ns, while the torus uniform run produced 25.14 ns. For the neighbouring pattern, both mesh and torus produced an average packet delay of 18.50 ns. These results show that the torus extension was integrated correctly and that it provides a small improvement for the chosen uniform mapping while giving the same behaviour as mesh under the local neighbouring pattern.**

***Index Terms*—Network-on-Chip, NoC, SystemC, wormhole routing, mesh topology, torus topology, router arbitration**

## I. Introduction

Modern systems-on-chip integrate many hardware blocks on the same die, and the communication method between those blocks has a direct effect on system behaviour. A shared bus is simple for small systems, but it becomes harder to scale once multiple modules need to communicate at the same time. A Network-on-Chip addresses that problem by treating on-chip communication as a packet-switched network. Data is packaged into packets, routers forward traffic hop-by-hop, and the topology determines how endpoints are connected.

The goal of this project was to model that behaviour in SystemC using a clocked, event-driven simulator. The work started from the provided 1x2 baseline and then followed the project schedule into a 4x4 mesh, the addition of delay measurement, the wormhole arbiter update, one-to-one traffic mappings, and finally a 4x4 torus extension. The final stage of the project was to verify that both topologies ran correctly under the required traffic patterns and then compare the results in a clear technical report. This report therefore serves as the completed version of the earlier interim submission and covers the full project rather than only the first milestone group.

## II. Theoretical Background

### A. Why a NoC is Used

A NoC can be viewed as a small communication network built inside a chip. Instead of forcing all traffic through one shared path, the design is divided across routers and links that move data from one block to another. This structure breaks the communication system into smaller parts, which makes larger systems easier to understand, verify, and extend. In simple terms, the NoC acts like a road network: sources inject traffic, routers decide the next direction, and sinks receive the data once the packet reaches its destination.

### B. Packets, Flits, and Wormhole Behaviour

In this simulator, communication is modeled as packets split into flits. A flit is the smallest unit moved through the router in one step. The header flit carries the routing information, while the body flits carry payload data and the tail flit marks the end of the packet. This matters because the route should not have to be recomputed for every body flit. After a header flit secures an output direction, the rest of that packet should follow the same path until the tail flit releases it.

That idea is the basis of wormhole routing. Instead of storing a full packet at every hop, the packet moves through the network as a stream of flits. The arbiter change completed in this project keeps an output reserved once a header flit wins access to that path. Only the tail flit is needed to clear the reservation. This better reflects realistic NoC flow control and reduces repeated route decisions inside the router.

### C. Mesh and Torus Topologies

The first full topology completed in the project was a 4x4 mesh. Each router has one local source/sink pair and up to four directional neighbours: north, south, east, and west. Routers on the boundary of the grid simply have fewer neighbours. The routing style for the mesh is based on XY forwarding, where the packet first moves across columns and then across rows until it reaches its destination.

The final extension was a 4x4 torus. A torus keeps the same local source/sink arrangement, but it adds wrap-around links at the boundaries. The top row connects back to the bottom row, and the left edge connects back to the right edge. This increases the number of available paths and can reduce the distance between some source-destination pairs. Because of that, torus behaviour is often different from mesh behaviour under longer or more distributed traffic patterns.

### D. Traffic Patterns and Metrics

Two one-to-one traffic patterns were used throughout the final experiments. In the uniform pattern, every source is mapped to a unique destination using a distributed permutation across the network. In the neighbouring pattern, each source communicates with a nearby node while still preserving the one-to-one requirement. These two cases were chosen because they exercise the network differently: one spreads traffic more widely and the other keeps traffic local.

The main performance metric used in the report is delay. Each packet is assigned an injection timestamp at the source,

and the sink later uses that information to measure delay. Two averages are recorded:

- average packet delay, measured when the tail flit of a packet arrives, and
- average flit delay, measured across all received flits.

In addition, the simulator records sent and received packet counts, sent and received flit counts, wrong-destination events, and a timeout-based deadlock check. These measurements are used to verify both correctness and stability.

## III. Work Progress

The project followed the original schedule closely and all planned milestone groups were completed. Table I summarizes the final status of the work before the detailed discussion of each stage.

TABLE I
Final Milestone Status Against the Original Schedule

| Week | Planned Work | Status |
|---|---|---|
| 1 | Build and understand the provided 1x2 baseline, module behaviour, tracing, and debug output | Done |
| 2 | Add send/receive timestamps, report average delay, and update the arbiter for wormhole-style route reservation | Done |
| 3 | Scale the design to a 4x4 mesh and verify basic sanity checks | Done |
| 4 | Add one-to-one uniform and neighbouring traffic mappings and collect initial performance data | Done |
| 5 | Add the 4x4 torus, compare mesh versus torus behaviour, and complete the final report | Done |

### A. Baseline Bring-Up

The first stage of the project was to run the supplied 1x2 design and use it as a reference model. This stage was less about adding new functionality and more about understanding how the SystemC modules interacted. In the baseline design, a source generates packets, a router forwards them, and a sink receives them. The baseline terminal output confirmed that all ten packets were delivered correctly from source 0 to sink 1.

This initial run was important because it exposed the handshake behaviour between modules and clarified how events were triggered in the simulation. It also established a stable starting point before the larger architectural changes were made.

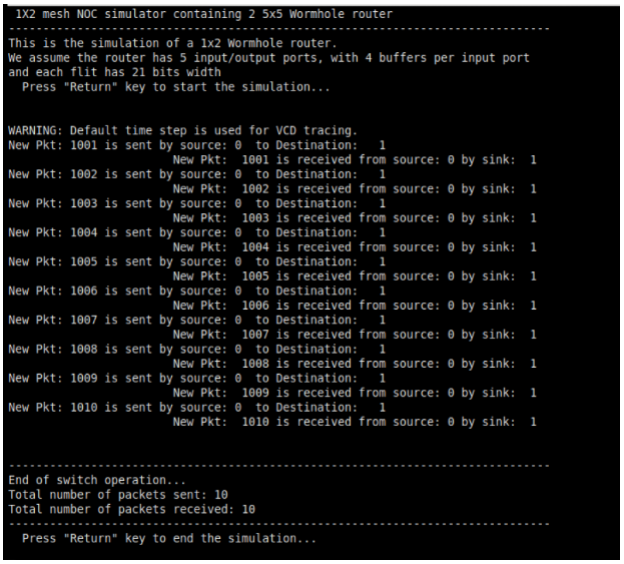



Fig. 1. Baseline 1x2 NoC terminal output showing successful delivery of all ten packets from source 0 to sink 1.

### B. Delay Measurement and Wormhole Arbitration

The second stage was the first major update to the original design. The packet structure was expanded so the source could record injection time and the sink could measure delay while also tracking packet and flit statistics. This turned the simulator from a correctness-only model into one that could report useful timing information.

The arbiter was also improved so that once a header flit is granted an output, that path stays reserved until the tail flit passes. This makes the router behaviour closer to wormhole routing and is one of the main technical improvements in the project. Together, these changes made the updated design more realistic and created the measurement framework used for all later comparisons.

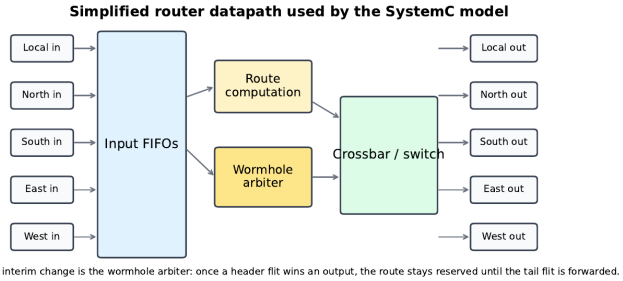



Fig. 2. Simplified router datapath used by the SystemC model. The wormhole arbiter keeps an output reserved until the tail flit is observed.

### C. Scaling to a 4x4 Mesh

After the router behaviour was updated, the top-level simulation was expanded from the 1x2 baseline into a 4x4 mesh. This required instantiating sixteen routers and connecting them with the correct directional links. Each router was also paired with one local source and one local sink. At this point, the

main challenge was no longer the behaviour of a single router, but the correctness of the full interconnection structure.

Once the mesh was connected, the simulation was checked using basic sanity conditions: packets sent versus packets received, flits sent versus flits received, destination correctness, and deadlock or timeout behaviour. Passing these checks was necessary before the result data could be treated as meaningful.

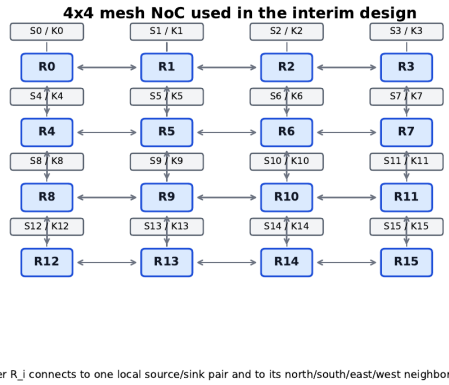



Fig. 3. 4x4 mesh NoC used in the completed design. Each router has one local source/sink pair and directional links to its neighbours when present.

### D. Traffic Mapping and Mesh Measurement

Once the mesh topology was stable, the two required traffic patterns were exercised. The uniform mapping spreads destinations across the network, while the neighbouring mapping keeps communication local. Both patterns completed successfully on the mesh and produced stable result data.

The mesh uniform run completed all 96 packets and 480 flits with zero wrong-destination flits and no timeout. The mesh neighbouring run also completed successfully and produced lower delay, which is expected because the destinations are only one hop away in that traffic pattern.

```
4x4 mesh NoC
Pattern: uniform
Packets/source: 6, flits/packet: 5

src  0 -> sink  3
src  1 -> sink  8
src  2 -> sink 13
src  3 -> sink  2
src  4 -> sink  7
src  5 -> sink 12
src  6 -> sink  1
src  7 -> sink  6
src  8 -> sink 11
src  9 -> sink  0
src 10 -> sink  5
src 11 -> sink 10
src 12 -> sink 15
src 13 -> sink  4
src 14 -> sink  9
src 15 -> sink 14
WARNING: Default time step is used for VCD tracing.

Results
Sim time:       118 ns
Packets sent:    96 / 96
Packets received:96 / 96
Flits sent:      480 / 480
Flits received:  480 / 480
Wrong destination flits: 0
Average packet delay (ns): 25.62
Average flit delay (ns):   21.62
Deadlock/timeout: NO
Per-sink one-to-one check: PASS
Overall sanity: PASS
```

Fig. 4. 4x4 mesh under the uniform one-to-one traffic pattern. The run completed successfully with correct delivery and measured packet and flit delays.

```
4x4 mesh NoC
Pattern: neighbour
Packets/source: 6, flits/packet: 5

src  0 -> sink  1
src  1 -> sink  0
src  2 -> sink  3
src  3 -> sink  2
src  4 -> sink  5
src  5 -> sink  4
src  6 -> sink  7
src  7 -> sink  6
src  8 -> sink  9
src  9 -> sink  8
src 10 -> sink 11
src 11 -> sink 10
src 12 -> sink 13
src 13 -> sink 12
src 14 -> sink 15
src 15 -> sink 14
WARNING: Default time step is used for VCD tracing.

Results
Sim time:       62 ns
Packets sent:    96 / 96
Packets received:96 / 96
Flits sent:      480 / 480
Flits received:  480 / 480
Wrong destination flits: 0
Average packet delay (ns): 18.50
Average flit delay (ns):   14.50
Deadlock/timeout: NO
Per-sink one-to-one check: PASS
Overall sanity: PASS
```

Fig. 5. 4x4 mesh under the neighbouring one-to-one traffic pattern. Because communication stays local, the measured delay is lower than in the uniform case.

### E. Torus Conversion and Final Validation

The final implementation stage was the torus conversion. The top-level design was updated so the boundary routers were no longer dead ends. Instead, wrap-around links were added from top to bottom and from left to right. This preserved the

same number of routers and endpoints while changing the set of available paths through the network.

The torus version required revisiting the route-decision logic so that the added wrap-around paths worked correctly with the existing wormhole reservation behaviour. After that update, the torus ran successfully for both required traffic patterns, with no timeout, no wrong-destination flits, and passing one-to-one checks.

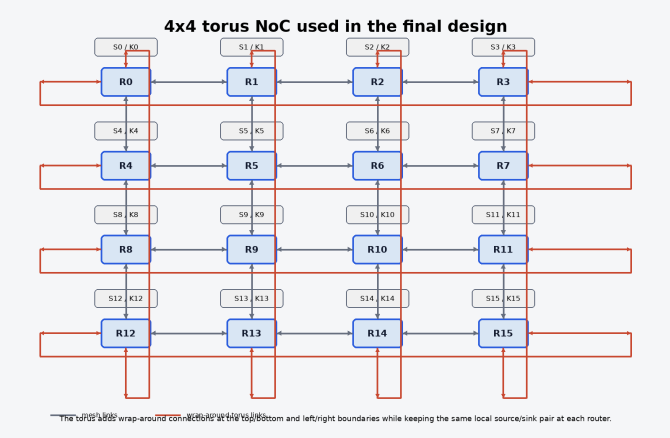



Fig. 6. 4x4 torus NoC used in the final design. Wrap-around links are added at the top/bottom and left/right boundaries.

```
4x4 torus NoC
Pattern: uniform
Packets/source: 6, flits/packet: 5

src  0 -> sink  3
src  1 -> sink  8
src  2 -> sink 13
src  3 -> sink  2
src  4 -> sink  7
src  5 -> sink 12
src  6 -> sink  1
src  7 -> sink  6
src  8 -> sink 11
src  9 -> sink  0
src 10 -> sink  5
src 11 -> sink 10
src 12 -> sink 15
src 13 -> sink  4
src 14 -> sink  9
src 15 -> sink 14
WARNING: Default time step is used for VCD tracing.

Results
Sim time:       118 ns
Packets sent:    96 / 96
Packets received:96 / 96
Flits sent:      480 / 480
Flits received:  480 / 480
Wrong destination flits: 0
Average packet delay (ns): 25.14
Average flit delay (ns):   21.14
Deadlock/timeout: NO
Per-sink one-to-one check: PASS
Overall sanity: PASS
```

Fig. 7. 4x4 torus under the uniform one-to-one traffic pattern. The run completed successfully and shows a small delay reduction relative to the mesh uniform case.

```
4x4 torus NoC
Pattern: neighbour
Packets/source: 6, flits/packet: 5

src  0 -> sink  1
src  1 -> sink  0
src  2 -> sink  3
src  3 -> sink  2
src  4 -> sink  5
src  5 -> sink  4
src  6 -> sink  7
src  7 -> sink  6
src  8 -> sink  9
src  9 -> sink  8
src 10 -> sink 11
src 11 -> sink 10
src 12 -> sink 13
src 13 -> sink 12
src 14 -> sink 15
src 15 -> sink 14
WARNING: Default time step is used for VCD tracing.

Results
Sim time:       62 ns
Packets sent:    96 / 96
Packets received:96 / 96
Flits sent:      480 / 480
Flits received:  480 / 480
Wrong destination flits: 0
Average packet delay (ns): 18.50
Average flit delay (ns):   14.50
Deadlock/timeout: NO
Per-sink one-to-one check: PASS
Overall sanity: PASS
```

Fig. 8. 4x4 torus under the neighbouring one-to-one traffic pattern. The measured result matches the mesh neighbouring case for the chosen local traffic mapping.

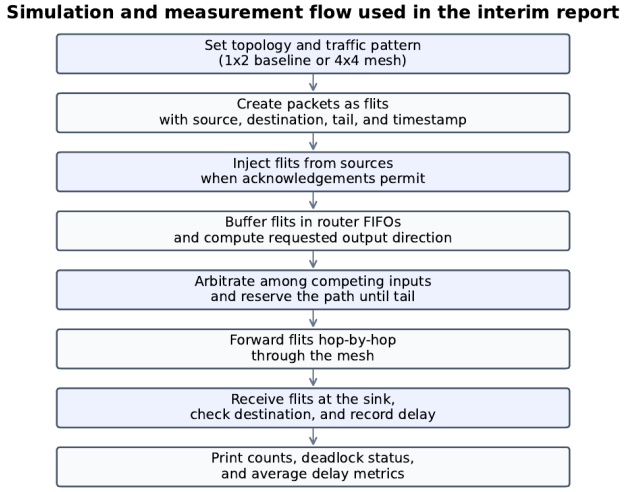



Fig. 9. High-level simulation and measurement flow used in the completed project.

## IV. Results and Discussion

Table II summarizes the final measured results across the four required runs. All four cases passed the basic correctness and timeout checks.

TABLE II
FINAL COMPARISON OF MESH AND TORUS RESULTS

| Topology | Pattern | Sim Time | Avg Packet | Avg Flit | Pass |
|---|---|---|---|---|---|
| Mesh | Uniform | 118 ns | 25.62 ns | 21.62 ns | Yes |
| Mesh | Neighbour | 62 ns | 18.50 ns | 14.50 ns | Yes |
| Torus | Uniform | 118 ns | 25.14 ns | 21.14 ns | Yes |
| Torus | Neighbour | 62 ns | 18.50 ns | 14.50 ns | Yes |

The first clear result is that all required runs completed successfully. Packet counts, flit counts, wrong-destination checks, and timeout checks all passed. This confirms that the final simulator is functionally stable for the tested mesh and torus cases.

The second result is that the neighbouring pattern produced the lowest delay in both topologies. That is expected because each source is communicating with a nearby destination, so packets do not need to travel far through the network. In this setup, the torus offers no practical improvement for the neighbouring case because the traffic is already local and the wrap-around links are not needed.

The third result is that the torus gives a small improvement in the uniform case. The measured average packet delay dropped from 25.62 ns in the mesh to 25.14 ns in the torus, and the average flit delay dropped from 21.62 ns to 21.14 ns. The difference is not large, but it is consistent with the purpose of the torus: some source-destination pairs can use shorter or less congested routes once wrap-around links are available.

It is also worth noting that the simulation time remained the same for both uniform runs and for both neighbouring runs. This means the main visible improvement in the torus is not the overall number of simulation cycles for this specific workload, but the average delay seen by the delivered traffic. For a larger design or more stressful traffic pattern, the gap between mesh and torus could become more noticeable.

Overall, the torus performed slightly better than the mesh in the more demanding uniform case, while both topologies behaved similarly in the neighbouring case. This suggests that the extra wrap-around links in the torus can improve routing efficiency, but they also increase the number of connections and possible paths in the network. As a result, the torus is more complex to implement and verify, and if the routing or arbitration is not handled carefully, it can be more vulnerable to deadlock than the simpler mesh topology.

## V. WORKS PENDING

The required implementation for the project is complete. Both the 4x4 mesh and the 4x4 torus have been implemented, the required traffic patterns have been exercised, and the comparison data has been collected. For that reason, there is no essential design work still pending for the final submission.

The only remaining work outside the scope of the completed project would be optional extensions. Examples include trying larger topologies, adding more traffic patterns, measuring throughput in addition to delay, or studying how the routing choice changes under heavier contention. These would be useful next steps, but they are no longer required to complete the current report.

## VI. TENTATIVE PLAN FOR PENDING WORK

Since the required project work has been completed, this section is best treated as a short note on future improvements rather than a plan for unfinished required tasks.

If the project were extended further, the next logical step would be to move beyond one-to-one traffic and test more demanding communication patterns. After that, the design could be scaled to a larger mesh or torus so the routing trade-offs become more obvious. A final improvement would be to add more detailed performance analysis, such as throughput or per-router congestion measurements, so the comparison is not based on delay alone.

## VII. CONCLUSION

This final stage completed the main goals planned for the NoC project. The provided 1x2 SystemC model was first used as a learning baseline, then extended with delay measurement and wormhole-style arbitration, scaled into a 4x4 mesh, and finally converted into a working 4x4 torus. Both required traffic mappings were tested on both topologies, and all four final runs passed the correctness and timeout checks.

The final results show that the torus extension works correctly and provides a small improvement for the selected uniform pattern, while the neighbouring pattern behaves the same in both topologies because the communication is already local. Most importantly, the project ended with a stable simulator that not only forwards traffic correctly, but also produces useful timing data that can support architectural comparison.

## APPENDIX A
## MODIFIED SOURCE FILES

The appendix below includes the main source and header files used in the final version of the project.

### A. `arbiter.cpp` and `arbiter.h`

These files implement the router arbitration logic and include the routing decision behaviour used for both the mesh and torus versions.

```
#undef SC_INCLUDE_FX

#include "packet.h"
#include "arbiter.h"

static const int K_ROWS = 4;
static const int K_COLS = 4;

static sc_uint<3> route_mesh(sc_uint<4> self,
    sc_uint<4> dest)
{
    int self_x = (int)(self % K_COLS);
    int self_y = (int)(self / K_COLS);
    int dest_x = (int)(dest % K_COLS);
    int dest_y = (int)(dest / K_COLS);

    if (dest_x > self_x) return 3;
    if (dest_x < self_x) return 5;
```

```
    if (dest_y > self_y) return 4;
    if (dest_y < self_y) return 2;
    return 1;
}

static sc_uint<3> route_torus_safe(sc_uint<4> self,
    sc_uint<4> src, sc_uint<4> dest)
{
    int self_x = (int)(self % K_COLS);
    int self_y = (int)(self / K_COLS);
    int src_x  = (int)(src % K_COLS);
    int src_y  = (int)(src / K_COLS);
    int dest_x = (int)(dest % K_COLS);
    int dest_y = (int)(dest / K_COLS);

    if (self == dest) {
        return 1;
    }

    if (self_x != src_x || self_y != src_y) {
        return route_mesh(self, dest);
    }

    int dx = dest_x - self_x;
    if (dx < 0) dx = -dx;
    int dy = dest_y - self_y;
    if (dy < 0) dy = -dy;

    int wrap_x = K_COLS - dx;
    int wrap_y = K_ROWS - dy;

    if (self_x == (K_COLS - 1) && dest_x < self_x &&
     wrap_x < dx) return 3;
    if (self_x == 0 && dest_x > self_x && wrap_x <
    dx) return 5;
    if (self_y == (K_ROWS - 1) && dest_y < self_y &&
     wrap_y < dy) return 4;
    if (self_y == 0 && dest_y > self_y && wrap_y <
    dy) return 2;

    return route_mesh(self, dest);
}

static bool is_output_free(sc_uint<5> free_mask,
    sc_uint<3> out)
{
    if (out < 1 || out > 5) return false;
    return free_mask[out - 1] == 1;
}

static void reserve_output(sc_uint<5>& free_mask,
    sc_uint<3> out)
{
    if (out >= 1 && out <= 5) {
        free_mask[out - 1] = 0;
    }
}

static void set_select(sc_uint<15>& v_select, int
    input_idx, sc_uint<3> out)
{
    int lo = input_idx * 3;
    int hi = lo + 2;
    v_select.range(hi, lo) = out;
}

void arbiter::func()
{
    bool connected[5];
    sc_uint<3> held_route[5];
    int reserved_owner[6];

    for (int i = 0; i < 5; i++) {
        connected[i] = false;
        held_route[i] = 0;
    }
    for (int i = 0; i < 6; i++) {
        reserved_owner[i] = 0;
    }

    while (true) {
        wait();

        sc_uint<15> v_select = 0;
        sc_uint<5> free_mask = 0;
        sc_uint<10> reqs[5];

        reqs[0] = req0.read();
        reqs[1] = req1.read();
        reqs[2] = req2.read();
        reqs[3] = req3.read();
        reqs[4] = req4.read();

        grant0.write(0);
        grant1.write(0);
        grant2.write(0);
        grant3.write(0);
        grant4.write(0);

        if (!free_out0.read()) free_mask[0] = 1;
        if (!free_out1.read()) free_mask[1] = 1;
        if (!free_out2.read()) free_mask[2] = 1;
        if (!free_out3.read()) free_mask[3] = 1;
        if (!free_out4.read()) free_mask[4] = 1;

        sc_uint<4> self_id = arbiter_id.read();
        bool use_torus = topology_torus.read();

        for (int i = 0; i < 5; i++) {
            sc_uint<10> rq = reqs[i];
            bool valid = rq[9];
            bool tail = rq[8];
            sc_uint<4> src = rq.range(7, 4);
            sc_uint<4> dest = rq.range(3, 0);

            if (!valid) {
                continue;
            }

            sc_uint<3> desired = connected[i]
                ? held_route[i]
                : (use_torus ? route_torus_safe(
    self_id, src, dest) : route_mesh(self_id, dest))
    ;

            if (desired < 1 || desired > 5) {
                continue;
            }

            bool owner_ok = (reserved_owner[desired]
     == 0) || (reserved_owner[desired] == (i + 1));
            bool free_ok = is_output_free(free_mask,
     desired);

            if (!owner_ok || !free_ok) {
                continue;
            }

            if (i == 0) grant0.write(1);
            if (i == 1) grant1.write(1);
            if (i == 2) grant2.write(1);
            if (i == 3) grant3.write(1);
            if (i == 4) grant4.write(1);

            set_select(v_select, i, desired);
            reserve_output(free_mask, desired);

            if (!connected[i]) {
```

```
                connected[i] = true;
                held_route[i] = desired;
                reserved_owner[desired] = i + 1;
            }

            if (tail) {
                connected[i] = false;
                held_route[i] = 0;
                reserved_owner[desired] = 0;
            }
        }

        aselect.write(v_select);
    }
}
```

Listing 1. Final `arbiter.cpp`

```
#include "systemc.h"

SC_MODULE(arbiter) {
    sc_in<sc_uint<4> > arbiter_id;
    sc_in<sc_uint<10> > req0;
    sc_in<sc_uint<10> > req1;
    sc_in<sc_uint<10> > req2;
    sc_in<sc_uint<10> > req3;
    sc_in<sc_uint<10> > req4;

    sc_in<bool> free_out0;
    sc_in<bool> free_out1;
    sc_in<bool> free_out2;
    sc_in<bool> free_out3;
    sc_in<bool> free_out4;
    sc_in<bool> topology_torus;

    sc_out<sc_uint<15> > aselect;

    sc_out<sc_uint<1> > grant0;
    sc_out<sc_uint<1> > grant1;
    sc_out<sc_uint<1> > grant2;
    sc_out<sc_uint<1> > grant3;
    sc_out<sc_uint<1> > grant4;

    sc_in<bool> aclk;

    void func();

    SC_CTOR(arbiter)
    {
        SC_THREAD(func);
        sensitive << aclk.neg();
    }
};
```

Listing 2. Final `arbiter.h`

### *B. buf_fifo.cpp and buf_fifo.h*

These files define the input buffer structure and generate the request format seen by the arbiter.

```
#include "buf_fifo.h"

static sc_uint<10> make_req(const fifo& q)
{
    sc_uint<10> v = 0;
    if (!q.empty) {
        v[9] = 1;
        v[8] = q.registers[0].h_t;
        v.range(7, 4) = q.registers[0].id;
        v.range(3, 0) = q.registers[0].dest;
    }
    return v;
}

void fifo::packet_in(const packet& data_packet)
{
    if (full) {
        return;
    }
    registers[reg_num++] = data_packet;
    empty = false;
    if (reg_num == 4) {
        full = true;
    }
}

packet fifo::packet_out()
{
    packet temp = registers[0];

    if (reg_num > 0) {
        reg_num--;
    }

    if (reg_num == 0) {
        empty = true;
    } else {
        for (int i = 0; i < reg_num; i++) {
            registers[i] = registers[i + 1];
        }
    }

    full = false;
    return temp;
}

void buf_fifo::func()
{
    fifo q0;
    packet b_temp;

    ack.write(false);
    req.write(make_req(q0));

    while (true) {
        wait();

        if (wr.event()) {
            q0.packet_in(wr.read());
            ack.write(q0.full);
            req.write(make_req(q0));
        }

        if (bclk.event()) {
            if (grant.read() == 1 && !q0.empty) {
                b_temp = q0.packet_out();
                re.write(b_temp);
            }
            ack.write(q0.full);
            req.write(make_req(q0));
        }
    }
}
```

Listing 3. Final `buf_fifo.cpp`

```
#include "packet.h"

SC_MODULE(buf_fifo) {
    sc_in<packet> wr;
    sc_out<packet> re;
    sc_in<sc_uint<1> > grant;
    sc_out<sc_uint<10> > req;
```

```
    sc_out<bool> ack;
    sc_in<bool> bclk;

    void func();

    SC_CTOR(buf_fifo)
    {
        SC_THREAD(func);
        sensitive << wr;
        sensitive << bclk.pos();
    }
};

struct fifo {
public:
    packet registers[4];
    bool full;
    bool empty;
    int reg_num;

    fifo()
    {
        full = false;
        empty = true;
        reg_num = 0;
    }

    void packet_in(const packet& data_packet);
    packet packet_out();
};
```

Listing 4. Final `buf_fifo.h`

### C. *crossbar.cpp and crossbar.h*

These files define the crossbar switching logic that forwards flits from router inputs to the selected outputs.

```
#include "packet.h"
#include "crossbar.h"

void crossbar::func()
{
    packet v_cross0;
    packet v_cross1;
    packet v_cross2;
    packet v_cross3;
    packet v_cross4;
    sc_uint<15> v_config;

    while (true) {
        wait();
        v_config = config.read();

        if (i0.event()) {
            v_cross0 = i0.read();
            switch (v_config.range(2, 0)) {
                case 1: o0.write(v_cross0); break;
                case 2: o1.write(v_cross0); break;
                case 3: o2.write(v_cross0); break;
                case 4: o3.write(v_cross0); break;
                case 5: o4.write(v_cross0); break;
                default: break;
            }
        }

        if (i1.event()) {
            v_cross1 = i1.read();
            switch (v_config.range(5, 3)) {
                case 1: o0.write(v_cross1); break;
                case 3: o2.write(v_cross1); break;
                case 4: o3.write(v_cross1); break;
                case 5: o4.write(v_cross1); break;
                default: break;
            }
        }

        if (i2.event()) {
            v_cross2 = i2.read();
            switch (v_config.range(8, 6)) {
                case 1: o0.write(v_cross2); break;
                case 2: o1.write(v_cross2); break;
                case 4: o3.write(v_cross2); break;
                case 5: o4.write(v_cross2); break;
                default: break;
            }
        }

        if (i3.event()) {
            v_cross3 = i3.read();
            switch (v_config.range(11, 9)) {
                case 1: o0.write(v_cross3); break;
                case 2: o1.write(v_cross3); break;
                case 3: o2.write(v_cross3); break;
                case 5: o4.write(v_cross3); break;
                default: break;
            }
        }

        if (i4.event()) {
            v_cross4 = i4.read();
            switch (v_config.range(14, 12)) {
                case 1: o0.write(v_cross4); break;
                case 2: o1.write(v_cross4); break;
                case 3: o2.write(v_cross4); break;
                case 4: o3.write(v_cross4); break;
                default: break;
            }
        }
    }
}
```

Listing 5. Final `crossbar.cpp`

```
#include "packet.h"

SC_MODULE(crossbar) {
    sc_in<packet> i0;
    sc_in<packet> i1;
    sc_in<packet> i2;
    sc_in<packet> i3;
    sc_in<packet> i4;

    sc_out<packet> o0;
    sc_out<packet> o1;
    sc_out<packet> o2;
    sc_out<packet> o3;
    sc_out<packet> o4;

    sc_in<sc_uint<15> > config;

    void func();

    SC_CTOR(crossbar)
    {
        SC_THREAD(func);
        sensitive << i0 << i1 << i2 << i3 << i4;
    }
};
```

Listing 6. Final `crossbar.h`

### D. *router.cpp and router.h*

These files define the overall router module and connect the FIFO, arbiter, and crossbar blocks.

```
#include "router.h"

void router::func()
{
    while (true) {
        wait();
        if (in0.event()) pkt_sent++;
        if (in1.event()) pkt_sent++;
        if (in2.event()) pkt_sent++;
        if (in3.event()) pkt_sent++;
        if (in4.event()) pkt_sent++;
    }
}
```

Listing 7. Final `router.cpp`

```
#include "packet.h"
#include "buf_fifo.h"
#include "crossbar.h"
#include "arbiter.h"

SC_MODULE(router) {
    sc_in<packet> in0;
    sc_in<packet> in1;
    sc_in<packet> in2;
    sc_in<packet> in3;
    sc_in<packet> in4;

    sc_in<sc_uint<4> > router_id;
    sc_in<bool> topology_torus;

    sc_out<packet> out0;
    sc_out<packet> out1;
    sc_out<packet> out2;
    sc_out<packet> out3;
    sc_out<packet> out4;

    sc_in<bool> inack0;
    sc_in<bool> inack1;
    sc_in<bool> inack2;
    sc_in<bool> inack3;
    sc_in<bool> inack4;

    sc_out<bool> outack0;
    sc_out<bool> outack1;
    sc_out<bool> outack2;
    sc_out<bool> outack3;
    sc_out<bool> outack4;

    sc_in<bool> rclk;

    buf_fifo* buf0;
    buf_fifo* buf1;
    buf_fifo* buf2;
    buf_fifo* buf3;
    buf_fifo* buf4;

    arbiter* arbiter0;
    crossbar* crossbar0;

    sc_signal<sc_uint<10> > req_s_0;
    sc_signal<sc_uint<10> > req_s_1;
    sc_signal<sc_uint<10> > req_s_2;
    sc_signal<sc_uint<10> > req_s_3;
    sc_signal<sc_uint<10> > req_s_4;

    sc_signal<sc_uint<15> > select_s;

    sc_signal<sc_uint<1> > gr_s_0;
    sc_signal<sc_uint<1> > gr_s_1;
    sc_signal<sc_uint<1> > gr_s_2;
    sc_signal<sc_uint<1> > gr_s_3;
    sc_signal<sc_uint<1> > gr_s_4;

    sc_signal<packet> re_s_0;
    sc_signal<packet> re_s_1;
    sc_signal<packet> re_s_2;
    sc_signal<packet> re_s_3;
    sc_signal<packet> re_s_4;

    int pkt_sent;
    void func();

    SC_CTOR(router)
    {
        buf0 = new buf_fifo("buf0");
        buf0->wr(in0); buf0->re(re_s_0); buf0->ack(
    outack0); buf0->req(req_s_0); buf0->grant(gr_s_0
    ); buf0->bclk(rclk);

        buf1 = new buf_fifo("buf1");
        buf1->wr(in1); buf1->re(re_s_1); buf1->ack(
    outack1); buf1->req(req_s_1); buf1->grant(gr_s_1
    ); buf1->bclk(rclk);

        buf2 = new buf_fifo("buf2");
        buf2->wr(in2); buf2->re(re_s_2); buf2->ack(
    outack2); buf2->req(req_s_2); buf2->grant(gr_s_2
    ); buf2->bclk(rclk);

        buf3 = new buf_fifo("buf3");
        buf3->wr(in3); buf3->re(re_s_3); buf3->ack(
    outack3); buf3->req(req_s_3); buf3->grant(gr_s_3
    ); buf3->bclk(rclk);

        buf4 = new buf_fifo("buf4");
        buf4->wr(in4); buf4->re(re_s_4); buf4->ack(
    outack4); buf4->req(req_s_4); buf4->grant(gr_s_4
    ); buf4->bclk(rclk);

        arbiter0 = new arbiter("arbiter0");
        arbiter0->arbiter_id(router_id);
        arbiter0->topology_torus(topology_torus);
        arbiter0->free_out0(inack0); arbiter0->
    free_out1(inack1); arbiter0->free_out2(inack2);
    arbiter0->free_out3(inack3); arbiter0->free_out4
    (inack4);
        arbiter0->req0(req_s_0); arbiter0->req1(
    req_s_1); arbiter0->req2(req_s_2); arbiter0->
    req3(req_s_3); arbiter0->req4(req_s_4);
        arbiter0->grant0(gr_s_0); arbiter0->grant1(
    gr_s_1); arbiter0->grant2(gr_s_2); arbiter0->
    grant3(gr_s_3); arbiter0->grant4(gr_s_4);
        arbiter0->aselect(select_s);
        arbiter0->aclk(rclk);

        crossbar0 = new crossbar("crossbar0");
        crossbar0->i0(re_s_0); crossbar0->i1(re_s_1)
    ; crossbar0->i2(re_s_2); crossbar0->i3(re_s_3);
    crossbar0->i4(re_s_4);
        crossbar0->o0(out0); crossbar0->o1(out1);
    crossbar0->o2(out2); crossbar0->o3(out3);
    crossbar0->o4(out4);
        crossbar0->config(select_s);

        SC_THREAD(func);
        sensitive << in0 << in1 << in2 << in3 << in4
    ;
        pkt_sent = 0;
    }
};
```

Listing 8. Final `router.h`

### *E. `source.cpp` and `source.h`*

These files define packet generation at each source, including destination mapping, packet timing, and packet/flit counting.

```
#include "source.h"

void source::func()
{
    packet v_packet_out;
    int flit_in_packet = 0;
    sc_uint<32> packet_start_cycle = 0;

    v_packet_out.data = 0;
    v_packet_out.pkt_clk = 0;
    v_packet_out.h_t = 0;
    v_packet_out.seq = 0;
    v_packet_out.inj_cycle = 0;

    while (true) {
        wait();

        if (pkt_snt >= total_packets) {
            continue;
        }

        if (!ach_in.read()) {
            if (flit_in_packet == 0) {
                packet_start_cycle = (sc_uint<32>)
    sc_time_stamp().value();
                v_packet_out.seq = current_seq;
            }

            v_packet_out.id = source_id.read();
            v_packet_out.dest = dest_id.read();
            v_packet_out.pkt_clk = ~v_packet_out.
    pkt_clk;
            v_packet_out.h_t = (flit_in_packet == (
    packet_len - 1)) ? 1 : 0;
            v_packet_out.inj_cycle =
    packet_start_cycle;
            v_packet_out.data = ((sc_uint<32>)
    source_id.read() << 24) |
                                ((sc_uint<32>)
    current_seq << 8) |
                                (sc_uint<32>)
    flit_in_packet;

            packet_out.write(v_packet_out);
            flit_snt++;
            flit_in_packet++;

            if (v_packet_out.h_t) {
                pkt_snt++;
                current_seq = current_seq + 1;
                flit_in_packet = 0;
            }
        }
    }
}
```

Listing 9. Final `source.cpp`

```
#include "packet.h"

SC_MODULE(source) {
    sc_out<packet> packet_out;
    sc_in<sc_uint<4> > source_id;
    sc_in<sc_uint<4> > dest_id;
    sc_in<bool> ach_in;
    sc_in_clk CLK;

    int pkt_snt;
    int flit_snt;
    int total_packets;
    int packet_len;
    sc_uint<16> current_seq;

    void func();

    SC_CTOR(source)
    {
        SC_CTHREAD(func, CLK.pos());
        pkt_snt = 0;
        flit_snt = 0;
        total_packets = 6;
        packet_len = 5;
        current_seq = 0;
    }
};
```

Listing 10. Final `source.h`

### *F. `sink.cpp` and `sink.h`*

These files define packet reception at each sink and measure delay, packet count, flit count, and wrong-destination events.

```
#include "sink.h"

void sink::receive_data()
{
    if (sclk.event()) {
        ack_out.write(false);
    }

    if (packet_in.event()) {
        packet v_packet = packet_in.read();
        double delay = (double)(sc_time_stamp().
    value() - v_packet.inj_cycle) / (double)sc_time
    (1, SC_NS).value();

        flit_recv++;
        ack_out.write(true);

        if (v_packet.dest != sink_id.read()) {
            wrong_dest++;
        }

        total_flit_delay += delay;

        if (v_packet.h_t) {
            pkt_recv++;
            total_packet_delay += delay;
        }
    }
}
```

Listing 11. Final `sink.cpp`

```
#include "packet.h"

SC_MODULE(sink) {
    sc_in<packet> packet_in;
    sc_out<bool> ack_out;
    sc_in<sc_uint<4> > sink_id;
    sc_in<bool> sclk;
```

```
    int pkt_recv;
    int flit_recv;
    int wrong_dest;
    double total_packet_delay;
    double total_flit_delay;

    void receive_data();

    SC_CTOR(sink)
    {
        SC_METHOD(receive_data);
        dont_initialize();
        sensitive << packet_in;
        sensitive << sclk.pos();
        pkt_recv = 0;
        flit_recv = 0;
        wrong_dest = 0;
        total_packet_delay = 0.0;
        total_flit_delay = 0.0;
    }
};
```

Listing 12. Final `sink.h`

### G. *packet.h*

This header defines the packet structure used across the NoC, including the fields needed for timing and tracing.

```
#ifndef PACKET
#define PACKET

#include "systemc.h"

struct packet {
    sc_uint<32> data;
    sc_uint<4>  id;
    sc_uint<4>  dest;
    sc_uint<1>  pkt_clk;
    sc_uint<1>  h_t;
    sc_uint<16> seq;
    sc_uint<32> inj_cycle;

    inline bool operator==(const packet& rhs) const
    {
        return rhs.data == data &&
               rhs.id == id &&
               rhs.dest == dest &&
               rhs.pkt_clk == pkt_clk &&
               rhs.h_t == h_t &&
               rhs.seq == seq &&
               rhs.inj_cycle == inj_cycle;
    }
};

inline ostream& operator<<(ostream& os, const packet
    & a)
{
    os << "pkt(data=" << a.data
       << ", id=" << a.id
       << ", dest=" << a.dest
       << ", seq=" << a.seq
       << ", tail=" << a.h_t
       << ", inj=" << a.inj_cycle << ")";
    return os;
}

inline void
#if defined(SC_API_VERSION_STRING)
sc_trace(sc_trace_file* tf, const packet& a, const
    std::string& name)
#else
sc_trace(sc_trace_file* tf, const packet& a, const
    sc_string& name)
#endif
{
    sc_trace(tf, a.data, name + ".data");
    sc_trace(tf, a.id, name + ".id");
    sc_trace(tf, a.dest, name + ".dest");
    sc_trace(tf, a.pkt_clk, name + ".pkt_clk");
    sc_trace(tf, a.h_t, name + ".h_t");
    sc_trace(tf, a.seq, name + ".seq");
    sc_trace(tf, a.inj_cycle, name + ".inj_cycle");
}

#endif
```

Listing 13. Final `packet.h`

### H. *main_noc.cpp*

This file defines the top-level NoC simulation and now supports both the mesh and torus topologies.

```
#include "systemc.h"
#include <iostream>
#include <iomanip>
#include <string>
#include "packet.h"
#include "source.h"
#include "sink.h"
#include "router.h"

static const int ROWS = 4;
static const int COLS = 4;
static const int NODES = ROWS * COLS;
static const int PACKETS_PER_SOURCE = 6;
static const int FLITS_PER_PACKET = 5;
static const sc_time CLK_PERIOD(1, SC_NS);
static const sc_time MAX_SIM_TIME(20000, SC_NS);

static int uniform_dest(int id)
{
    return (id * 5 + 3) % NODES;
}

static int neighbour_dest(int id)
{
    int row = id / COLS;
    int col = id % COLS;
    if ((col % 2) == 0) return row * COLS + col + 1;
    return row * COLS + col - 1;
}

static int expected_per_sink()
{
    return PACKETS_PER_SOURCE;
}

static int total_packets_received(sink* sinks[])
{
    int total = 0;
    for (int i = 0; i < NODES; i++) total += sinks[i
    ]->pkt_recv;
    return total;
}

static int total_flits_received(sink* sinks[])
{
    int total = 0;
    for (int i = 0; i < NODES; i++) total += sinks[i
    ]->flit_recv;
    return total;
}
```

```
static int index_of(int row, int col)
{
    return row * COLS + col;
}

static int mod_wrap(int value, int size)
{
    int out = value % size;
    if (out < 0) out += size;
    return out;
}

static std::string link_name(const std::string& base
    , int id)
{
    return base + "_" + std::to_string(id);
}

int sc_main(int argc, char* argv[])
{
    std::string topology = "mesh";
    std::string pattern = "uniform";

    if (argc > 1) {
        std::string arg1 = argv[1];
        if (arg1 == "mesh" || arg1 == "torus") {
            topology = arg1;
            if (argc > 2) {
                pattern = argv[2];
            }
        } else {
            pattern = arg1;
        }
    }

    if (pattern == "neighbor") {
        pattern = "neighbour";
    }

    if ((topology != "mesh" && topology != "torus")
    ||
        (pattern != "uniform" && pattern != "
    neighbour")) {
        std::cout << "Usage: ./noc.x [mesh|torus] [
    uniform|neighbour]" << std::endl;
        return 1;
    }

    bool use_torus = (topology == "torus");

    sc_signal<packet> src_to_router[NODES];
    sc_signal<packet> router_to_sink[NODES];
    sc_signal<bool> ack_src[NODES];
    sc_signal<bool> ack_sink[NODES];
    sc_signal<sc_uint<4> > node_sig[NODES];
    sc_signal<sc_uint<4> > dest_sig[NODES];
    sc_signal<bool> topology_sig[NODES];

    sc_signal<packet> link_north[NODES];
    sc_signal<packet> link_east[NODES];
    sc_signal<packet> link_south[NODES];
    sc_signal<packet> link_west[NODES];

    sc_signal<bool> ack_north[NODES];
    sc_signal<bool> ack_east[NODES];
    sc_signal<bool> ack_south[NODES];
    sc_signal<bool> ack_west[NODES];

    sc_signal<packet> dummy_pkt[NODES][4];
    sc_signal<bool> dummy_inack[NODES][4];
    sc_signal<bool> dummy_outack[NODES][4];

    sc_clock s_clock("S_CLOCK", CLK_PERIOD);
    sc_clock r_clock("R_CLOCK", CLK_PERIOD);
    sc_clock d_clock("D_CLOCK", CLK_PERIOD);

    source* sources[NODES];
    sink* sinks[NODES];
    router* routers[NODES];

    std::string trace_base = "graph_" + topology + "
    _" + pattern;
    sc_trace_file* tf = sc_create_vcd_trace_file(
    trace_base.c_str());
    sc_trace(tf, s_clock, "s_clock");
    sc_trace(tf, r_clock, "r_clock");
    sc_trace(tf, d_clock, "d_clock");

    for (int i = 0; i < NODES; i++) {
        std::string sname = "source" + std::
    to_string(i);
        std::string kname = "sink" + std::to_string(
    i);
        std::string rname = "router" + std::
    to_string(i);

        sources[i] = new source(sname.c_str());
        sinks[i] = new sink(kname.c_str());
        routers[i] = new router(rname.c_str());

        sources[i]->packet_out(src_to_router[i]);
        sources[i]->source_id(node_sig[i]);
        sources[i]->dest_id(dest_sig[i]);
        sources[i]->ach_in(ack_src[i]);
        sources[i]->CLK(s_clock);
        sources[i]->total_packets =
    PACKETS_PER_SOURCE;
        sources[i]->packet_len = FLITS_PER_PACKET;

        sinks[i]->packet_in(router_to_sink[i]);
        sinks[i]->ack_out(ack_sink[i]);
        sinks[i]->sink_id(node_sig[i]);
        sinks[i]->sclk(d_clock);

        routers[i]->in0(src_to_router[i]);
        routers[i]->out0(router_to_sink[i]);
        routers[i]->inack0(ack_sink[i]);
        routers[i]->outack0(ack_src[i]);
        routers[i]->router_id(node_sig[i]);
        routers[i]->topology_torus(topology_sig[i]);
        routers[i]->rclk(r_clock);

        topology_sig[i].write(use_torus);
    }

    for (int row = 0; row < ROWS; row++) {
        for (int col = 0; col < COLS; col++) {
            int id = index_of(row, col);

            int north_row = row - 1;
            int south_row = row + 1;
            int east_col = col + 1;
            int west_col = col - 1;

            bool has_north = use_torus || (north_row
     >= 0);
            bool has_south = use_torus || (south_row
     < ROWS);
            bool has_east = use_torus || (east_col <
     COLS);
            bool has_west = use_torus || (west_col
    >= 0);

            if (use_torus) {
                north_row = mod_wrap(north_row, ROWS
    );
                south_row = mod_wrap(south_row, ROWS
    );
```

```
            east_col = mod_wrap(east_col, COLS);
            west_col = mod_wrap(west_col, COLS);
        }

        if (has_north) {
            int north_id = index_of(north_row,
col);
            routers[id]->in1(link_south[north_id
]);
            routers[id]->outack1(ack_south[
north_id]);
            routers[id]->out1(link_north[id]);
            routers[id]->inack1(ack_north[id]);
        } else {
            routers[id]->in1(dummy_pkt[id][0]);
            routers[id]->outack1(dummy_outack[id
][0]);
            routers[id]->out1(dummy_pkt[id][1]);
            routers[id]->inack1(dummy_inack[id
][0]);
        }

        if (has_east) {
            int east_id = index_of(row, east_col
);
            routers[id]->in2(link_west[east_id])
;
            routers[id]->outack2(ack_west[
east_id]);
            routers[id]->out2(link_east[id]);
            routers[id]->inack2(ack_east[id]);
        } else {
            routers[id]->in2(dummy_pkt[id][2]);
            routers[id]->outack2(dummy_outack[id
][1]);
            routers[id]->out2(dummy_pkt[id][3]);
            routers[id]->inack2(dummy_inack[id
][1]);
        }

        if (has_south) {
            int south_id = index_of(south_row,
col);
            routers[id]->in3(link_north[south_id
]);
            routers[id]->outack3(ack_north[
south_id]);
            routers[id]->out3(link_south[id]);
            routers[id]->inack3(ack_south[id]);
        } else {
            routers[id]->in3(dummy_pkt[id][0]);
            routers[id]->outack3(dummy_outack[id
][2]);
            routers[id]->out3(dummy_pkt[id][1]);
            routers[id]->inack3(dummy_inack[id
][2]);
        }

        if (has_west) {
            int west_id = index_of(row, west_col
);
            routers[id]->in4(link_east[west_id])
;
            routers[id]->outack4(ack_east[
west_id]);
            routers[id]->out4(link_west[id]);
            routers[id]->inack4(ack_west[id]);
        } else {
            routers[id]->in4(dummy_pkt[id][2]);
            routers[id]->outack4(dummy_outack[id
][3]);
            routers[id]->out4(dummy_pkt[id][3]);
            routers[id]->inack4(dummy_inack[id
][3]);
        }
    }
}

for (int i = 0; i < NODES; i++) {
    sc_trace(tf, src_to_router[i], link_name("
src_to_router", i));
    sc_trace(tf, router_to_sink[i], link_name("
router_to_sink", i));
    sc_trace(tf, ack_src[i], link_name("ack_src"
, i));
    sc_trace(tf, ack_sink[i], link_name("
ack_sink", i));
    sc_trace(tf, node_sig[i], link_name("node_id
", i));
    sc_trace(tf, dest_sig[i], link_name("dest_id
", i));
    sc_trace(tf, topology_sig[i], link_name("
topology_torus", i));
    sc_trace(tf, link_north[i], link_name("
link_north", i));
    sc_trace(tf, link_east[i], link_name("
link_east", i));
    sc_trace(tf, link_south[i], link_name("
link_south", i));
    sc_trace(tf, link_west[i], link_name("
link_west", i));
    sc_trace(tf, ack_north[i], link_name("
ack_north", i));
    sc_trace(tf, ack_east[i], link_name("
ack_east", i));
    sc_trace(tf, ack_south[i], link_name("
ack_south", i));
    sc_trace(tf, ack_west[i], link_name("
ack_west", i));
}

std::cout << std::endl;
std::cout << "4x4 " << topology << " NoC" << std
::endl;
std::cout << "Pattern: " << pattern << std::endl
;
std::cout << "Packets/source: " <<
PACKETS_PER_SOURCE << ", flits/packet: " <<
FLITS_PER_PACKET << std::endl;
std::cout << std::endl;

for (int i = 0; i < NODES; i++) {
    int dest = (pattern == "uniform") ?
uniform_dest(i) : neighbour_dest(i);
    node_sig[i].write(i);
    dest_sig[i].write(dest);
    std::cout << "src " << std::setw(2) << i <<
" -> sink " << std::setw(2) << dest << std::endl
;
}

int expected_packets = NODES *
PACKETS_PER_SOURCE;
int expected_flits = expected_packets *
FLITS_PER_PACKET;
bool timeout = false;

while (total_packets_received(sinks) <
expected_packets) {
    if (sc_time_stamp() >= MAX_SIM_TIME) {
        timeout = true;
        break;
    }
    sc_start(CLK_PERIOD);
}

int sent_packets = 0;
int sent_flits = 0;
```

```
    int recv_packets = 0;
    int recv_flits = 0;
    int wrong_dest = 0;
    double total_packet_delay = 0.0;
    double total_flit_delay = 0.0;
    bool sink_balance_ok = true;

    for (int i = 0; i < NODES; i++) {
        sent_packets += sources[i]->pkt_snt;
        sent_flits += sources[i]->flit_snt;
        recv_packets += sinks[i]->pkt_recv;
        recv_flits += sinks[i]->flit_recv;
        wrong_dest += sinks[i]->wrong_dest;
        total_packet_delay += sinks[i]->
    total_packet_delay;
        total_flit_delay += sinks[i]->
    total_flit_delay;
        if (sinks[i]->pkt_recv != expected_per_sink
    ()) {
            sink_balance_ok = false;
        }
    }

    std::cout << std::endl;
    std::cout << "Results" << std::endl;
    std::cout << "Sim time:       " << sc_time_stamp
    () << std::endl;
    std::cout << "Packets sent:    " << sent_packets
     << " / " << expected_packets << std::endl;
    std::cout << "Packets received:" << recv_packets
     << " / " << expected_packets << std::endl;
    std::cout << "Flits sent:      " << sent_flits
    << " / " << expected_flits << std::endl;
    std::cout << "Flits received:  " << recv_flits
    << " / " << expected_flits << std::endl;
    std::cout << "Wrong destination flits: " <<
    wrong_dest << std::endl;

    if (recv_packets > 0) {
        std::cout << std::fixed << std::setprecision
    (2);
        std::cout << "Average packet delay (ns): "
    << (total_packet_delay / recv_packets) << std::
    endl;
    }
    if (recv_flits > 0) {
        std::cout << std::fixed << std::setprecision
    (2);
        std::cout << "Average flit delay (ns):   "
    << (total_flit_delay / recv_flits) << std::endl;
    }

    bool pass = (!timeout) &&
                (sent_packets == expected_packets)
    &&
                (recv_packets == expected_packets)
    &&
                (sent_flits == expected_flits) &&
                (recv_flits == expected_flits) &&
                (wrong_dest == 0) &&
                sink_balance_ok;

    std::cout << "Deadlock/timeout: " << (timeout ?
    "YES" : "NO") << std::endl;
    std::cout << "Per-sink one-to-one check: " << (
    sink_balance_ok ? "PASS" : "FAIL") << std::endl;
    std::cout << "Overall sanity: " << (pass ? "PASS
    " : "FAIL") << std::endl;

    sc_close_vcd_trace_file(tf);
    return pass ? 0 : 2;
}
```

Listing 14. Final `main_noc.cpp`

## REFERENCES


[1] L. Benini and G. De Micheli, "Networks on chips: A new SoC paradigm," *Computer*, vol. 35, no. 1, pp. 70–78, Jan. 2002.

[2] G. Khan, "NOC: Networks on Chip SoC Interconnection Structures," COE838/EE8221 *Computer*, pp. 22–27, Jan. 2026.